

LungInsight Imaging Pipeline Architecture

The architecture should preserve the current **pretrained ViTDet3D detector in Stage 04** as the primary lesion/candidate proposal mechanism. The pipeline should therefore be organized around a strict spatial-data contract rather than replacing detection with LoG or another handcrafted detector.

The core coordinate convention throughout the pipeline is:

All voxel coordinates: (z, y, x)

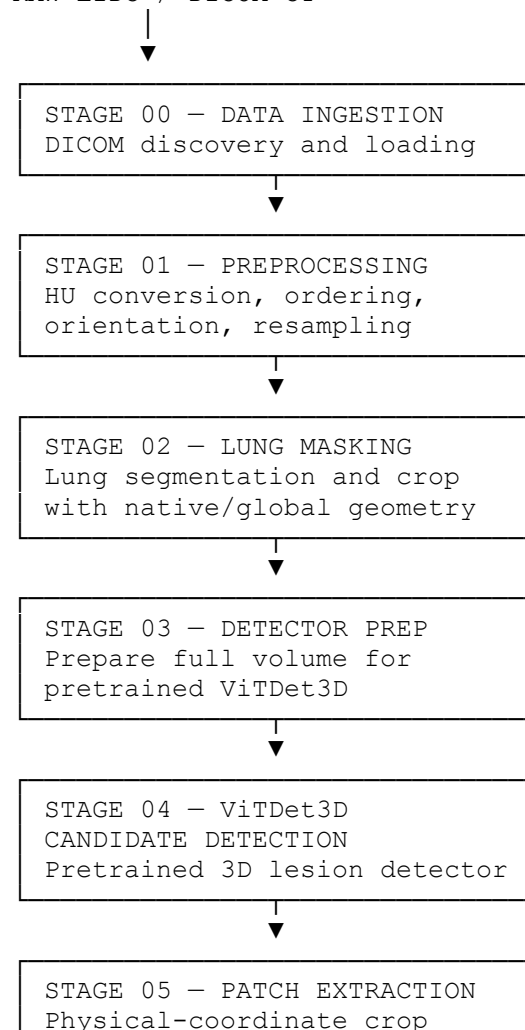
All array volumes: (D, H, W) = (Z, Y, X)

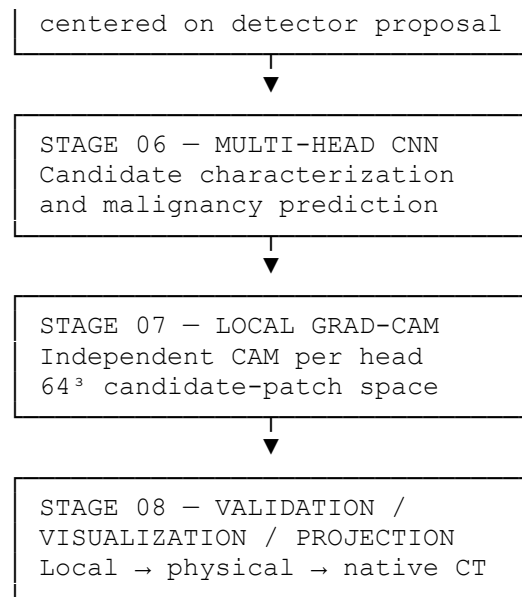
PyTorch volumetric tensors: (B, C, Z, Y, X)

1. Pipeline Stages & Responsibilities

Overall architecture

RAW LIDC / DICOM CT





Stage 00 — DICOM Ingestion

Input

DICOM series

Responsibilities

- Discover all slices belonging to the CT series.
- Sort slices by physical position.
- Validate orientation consistency.
- Convert pixel values into Hounsfield Units.
- Extract patient/series metadata.
- Establish the initial physical coordinate system.

Canonical output

```
volume_hu.shape  
# (Z, Y, X)
```

Example:

```
(300, 512, 512)
```

Metadata:

```
{  
  "patient_id": "LIDC-IDRI-0141",  
  "coordinate_order": "ZYX",  
  "volume_shape_zyx": [300, 512, 512],  
  "spacing_zyx_mm": [2.5, 0.68, 0.68],  
  "origin_xyz_mm": [...],  
}
```

```
"orientation": "...",
"space": "dicom_native"
}
```

The original DICOM geometry should always remain recoverable.

Stage 01 — CT Preprocessing and Resampling

This stage creates the canonical CT representation used by downstream processing.

Operations

1. Resample

Resample to isotropic spacing, for example:

`1.0 × 1.0 × 1.0 mm`

using interpolation appropriate for CT intensities.

2. HU clipping

For example:

`[-1000 HU, +400 HU]`

3. Preserve geometry

The original and resampled geometry must both be stored.

Output

`ct_resampled_hu`

Shape:

`(Z, Y, X)`

Example:

`(450, 350, 350)`

Record:

```
{
  "space": "stage01_resampled_ct",
```

```
"coordinate_order": "ZYX",  
"shape_zyx": [450, 350, 350],  
"spacing_zyx_mm": [1.0, 1.0, 1.0],  
"intensity_space": "HU",  
"hu_clip": [-1000, 400],  
"source_space": "dicom_native"  
}
```

Stage 02 — Lung Segmentation and Spatial Context

Stage 02 produces the lung mask and establishes the authoritative global CT volume used by later visualization.

Operations

```
Stage 01 CT  
├── Lung segmentation  
├── Mask cleanup  
├── Connected-component filtering  
└── Optional lung-region crop
```

Outputs

```
ct_native_zyx  
lung_mask_zyx
```

Both must have:

```
(Z, Y, X)
```

Important requirement

If the CT is cropped, the crop transform must be explicit.

For example:

```
{  
  "source_shape_zyx": [450, 350, 350],  
  "crop_start_zyx": [20, 15, 10],  
  "crop_end_zyx": [430, 330, 340],  
}
```

```

    "output_shape_zyx": [410, 315, 330],

    "spacing_zyx_mm": [1.0, 1.0, 1.0],

    "coordinate_order": "ZYX"
}

```

The transformation must always be reconstructable:

```

global_voxel_zyx =
crop_start_zyx + local_voxel_zyx

```

Stage 03 — ViTDet3D Input Preparation

This stage should prepare the Stage 02 CT for the **pretrained ViTDet3D detector** without changing the detector's spatial meaning.

Responsibilities

- Apply exactly the normalization expected by the pretrained detector.
- Handle detector-required padding.
- Handle sliding windows or tiling if required.
- Preserve the mapping:

```

ViTDet3D input coordinates
      ↓
Stage 02 global CT coordinates

```

Input

ct_stage02

Shape:

(Z, Y, X)

Detector tensor

detector_input

Shape:

(B, C, Z, Y, X)

Typically:

(1, 1, Z, Y, X)

or detector-specific tiled windows:

(N, C, Dz, Dy, Dx)

Critical metadata

```
{
  "detector_model": "ViTDet3D",
  "coordinate_order": "ZYX",

  "source_space": "stage02_native_ct",

  "input_shape_zyx": [Z, Y, X],

  "padding_before_zyx": [0, 0, 0],

  "padding_after_zyx": [0, 0, 0],

  "normalization": "detector-specific"
}
```

This stage must not silently transpose:

ZYX ↔ XYZ

Stage 04 — Pretrained ViTDet3D Candidate Detection

This is the primary candidate proposal stage.

The architecture should preserve:

Stage 04 = pretrained ViTDet3D lesion/nodule detection

It should not be replaced by LoG, handcrafted blob detection, or heuristic candidate generation.

Those techniques may be retained only as:

- diagnostic tools,
- fallback analysis,
- research comparison baselines.

Responsibilities

The model receives the Stage 03 volume and produces lesion proposals.

A proposal should minimally contain:

```
candidate_id
detector confidence
center Z,Y,X
```

bounding box
physical dimensions
detector metadata

Recommended record:

```
{
  "candidate_id": 17,

  "source": "ViTDet3D",

  "detector_score": 0.947,

  "coordinate_order": "ZYX",

  "center_zyx": [
    182.4,
    211.7,
    143.2
  ],

  "bbox_start_zyx": [
    172,
    200,
    132
  ],

  "bbox_end_zyx": [
    194,
    224,
    156
  ],

  "bbox_size_zyx_voxels": [
    22,
    24,
    24
  ],

  "spacing_zyx_mm": [
    1.0,
    1.0,
    1.0
  ],

  "space": "stage02_native_ct",

  "model": {
    "name": "ViTDet3D",
    "checkpoint": "...",
    "model_version": "..."
  }
}
```

Recommended rule

The detector center becomes the authoritative patch center:

Stage 04 center

↓
Stage 05 crop center
↓
Stage 06 classifier patch center
↓
Stage 07 Grad-CAM local center
↓
Stage 08 projected lesion location

This prevents the previous class of bugs where Stage 05 silently recenters on a bounding-box midpoint instead of the actual detector proposal.

Stage 05 — Physical Candidate Patch Extraction

This stage converts a global Stage 02 lesion location into a fixed classifier patch.

The important design principle is:

Crop in physical space, then sample directly into the classifier grid.

Do not:

native crop
↓
arbitrary resize
↓
classifier patch

because that can introduce spatial offsets and inconsistent physical fields of view.

Input

Stage 04 candidate
+
Stage 02 CT

The detector center is:

candidate_center_zyx

For example:

[182.4, 211.7, 143.2]

Physical sampling model

For local patch coordinate:

```
l = (lz, ly, lx)
```

and local center:

```
c_local = (31.5, 31.5, 31.5)
```

the corresponding physical coordinate is:

```
physical_zyx_mm =  
patch_center_physical_zyx_mm  
+  
(local_zyx - local_center_zyx)  
*  
patch_spacing_zyx_mm
```

Then that physical location is mapped back into the Stage 02 CT for interpolation.

Output patch

Classifier patch:

```
patch_normalized.shape
```

must be:

```
(64, 64, 64)
```

with:

```
dtype = float32
```

Normalization:

```
normalized = (HU + 1000) / 1400
```

Expected range:

```
[0, 1]
```

PyTorch classifier input:

```
(B, C, Z, Y, X)
```

therefore:

(1, 1, 64, 64, 64)

Recommended Stage 05 record

```
{
  "candidate_id": 17,

  "coordinate_order": "ZYX",

  "source": "ViTDet3D",

  "candidate_center_zyx": [
    182.4,
    211.7,
    143.2
  ],

  "patch_shape_zyx": [
    64,
    64,
    64
  ],

  "local_patch_center_zyx": [
    31.5,
    31.5,
    31.5
  ],

  "patch_spacing_zyx_mm": [
    1.0,
    1.0,
    1.0
  ],

  "patch_fov_zyx_mm": [
    64.0,
    64.0,
    64.0
  ],

  "patch_center_physical_zyx_mm": [
    ...
  ],

  "native_spacing_zyx_mm": [
    1.0,
    1.0,
    1.0
  ],

  "native_volume_shape_zyx": [
    ...
  ],

  "patch_file": "patches/candidate_17.npy",

  "dtype": "float32",
```

```
"normalization": "(HU + 1000) / 1400",  
"value_range": "[0,1]",  
"source_space": "stage02_native_ct",  
"geometry_authority": "stage05"  
}
```

Stage 06 — Multi-Head Candidate Classification

Stage 06 receives one independent 64^3 patch for every Stage 04 candidate.

Input

```
(1, 1, 64, 64, 64)
```

No additional HU normalization should occur if Stage 05 has already produced:

```
(HU + 1000) / 1400
```

Multi-head outputs

The current classifier heads are:

```
calcification  
lobulation  
malignancy  
margin  
sphericity  
spiculation  
subtlety  
texture
```

The output should preserve the candidate identity and Stage 05 geometry.

Recommended classification record:

```
{  
  "candidate_id": 17,  
  
  "patch_file": "patches/candidate_17.npy",  
  
  "coordinate_order": "ZYX",  
  
  "predictions": {  
    "calcification": 0.21,  
    "lobulation": 0.78,  
    "malignancy": 0.64,  
  },  
}
```

```

    "margin": 0.59,
    "sphericity": 0.33,
    "spiculation": 0.71,
    "subtlety": 0.81,
    "texture": 0.48
  },

  "candidate_center_zyx": [
    182.4,
    211.7,
    143.2
  ],

  "patch_shape_zyx": [
    64,
    64,
    64
  ],

  "patch_spacing_zyx_mm": [
    1.0,
    1.0,
    1.0
  ],

  "geometry_authority": "stage05"
}

```

Stage 06 should ideally preserve the complete Stage 05 spatial context instead of reconstructing it.

Stage 07 — Candidate-Level Multi-Head Grad-CAM

Stage 07 generates explainability maps.

The critical rule is:

Stage 07 operates only in local candidate patch space.

It must not project directly into the full CT.

Input

Stage 05 normalized patch

Shape:

(64, 64, 64)

Classifier tensor:

```
(1, 1, 64, 64, 64)
```

Independent CAM generation

Each head receives an independent:

```
forward pass  
+  
backward pass
```

Therefore:

```
calcification.npy  
lobulation.npy  
malignancy.npy  
margin.npy  
sphericity.npy  
spiculation.npy  
subtlety.npy  
texture.npy
```

Each file must be:

```
shape = (64, 64, 64)  
dtype = float32  
coordinate order = Z,Y,X  
value range  $\approx$  [0,1]
```

There should be no:

```
shared full-volume CAM  
cross-head averaging  
candidate accumulation
```

Stage 07 spatial contract

```
GLOBAL SPACE  
Stage 02 CT  
    |  
    | candidate center  
    ▼  
PHYSICAL PATCH SPACE  
continuous millimetres  
    |  
    | Stage 05 sampling transform  
    ▼  
LOCAL PATCH SPACE  
64 × 64 × 64  
    |  
    ▼  
GRAD-CAM SPACE
```

64 × 64 × 64

The Grad-CAM array is therefore directly aligned with the classifier patch:

```
patch[z, y, x]
↓
gradcam[z, y, x]
```

Stage 08 — Local-to-Global Projection and Visualization

Stage 08 is responsible for the spatial transformation that Stage 07 deliberately does not perform.

Input

```
Stage 02 CT
+
Stage 05 geometry
+
Stage 07 local Grad-CAM
```

The mapping is:

```
local patch
↓
physical coordinates
↓
Stage 02 native/global CT
```

Mathematically:

```
physical_zyx_mm =
patch_center_physical_zyx_mm
+
(local_zyx - local_patch_center_zyx)
*
patch_spacing_zyx_mm
```

Then:

```
native_zyx =
candidate_center_zyx
+
(local_zyx - local_patch_center_zyx)
*
patch_spacing_zyx_mm
/
native_spacing_zyx_mm
```

assuming both spaces have compatible origins and orientation.

If the Stage 02 volume includes crop offsets, origin and crop transforms must be explicitly incorporated.

2. Canonical Data Record Specifications

A. Volumetric CT Record

Recommended structure:

```
stage02/  
  ct_hu.npy  
  lung_mask.npy  
  metadata.json
```

CT

```
ct_hu.npy  
shape: (Z,Y,X)  
dtype: float32  
space: HU
```

Lung mask

```
lung_mask.npy  
shape: (Z,Y,X)  
dtype: uint8  
values: {0,1}
```

Metadata

```
{  
  "patient_id": "LIDC-IDRI-0141",  
  
  "coordinate_order": "ZYX",  
  
  "space": "stage02_native_ct",  
  
  "volume_shape_zyx": [450, 350, 350],  
  
  "spacing_zyx_mm": [1.0, 1.0, 1.0],  
  
  "intensity": {  
    "space": "HU",  
    "clip_range": [-1000, 400]  
  },  
  
  "transforms": {  
    "dicom_to_stage01": "...",  
    "stage01_to_stage02": "..."  
  }  
}
```

B. ViTDet3D Candidate Record

A candidate record should be self-contained enough for Stage 05.

```
{
  "candidate_id": 17,

  "detector": {
    "name": "ViTDet3D",
    "checkpoint": "...",
    "score": 0.947
  },

  "coordinate_order": "ZYX",

  "space": "stage02_native_ct",

  "center_zyx": [
    182.4,
    211.7,
    143.2
  ],

  "bbox_start_zyx": [
    172,
    200,
    132
  ],

  "bbox_end_zyx": [
    194,
    224,
    156
  ],

  "volume_shape_zyx": [
    450,
    350,
    350
  ],

  "spacing_zyx_mm": [
    1.0,
    1.0,
    1.0
  ]
}
```

C. Classifier Patch

Physical file:

candidate_17.npy

Array:

```
dtype: float32
shape: (64,64,64)
order: Z,Y,X
range: [0,1]
```

Tensor form:

```
(1,1,64,64,64)
```

Normalization:

```
normalized = (HU + 1000) / 1400
```

D. Patch Geometry Record

This is one of the most important records in the entire system.

```
{
  "candidate_id": 17,

  "coordinate_order": "ZYX",

  "candidate_center_zyx": [
    182.4,
    211.7,
    143.2
  ],

  "native_volume_shape_zyx": [
    450,
    350,
    350
  ],

  "native_spacing_zyx_mm": [
    1.0,
    1.0,
    1.0
  ],

  "patch_shape_zyx": [
    64,
    64,
    64
  ],

  "local_patch_center_zyx": [
    31.5,
    31.5,
    31.5
  ],

  "patch_center_physical_zyx_mm": [
    ...
  ],
```

```

    "patch_spacing_zyx_mm": [
        1.0,
        1.0,
        1.0
    ],

    "patch_fov_zyx_mm": [
        64.0,
        64.0,
        64.0
    ],

    "geometry_authority": "stage05"
}

```

This record should be propagated unchanged through Stages 06, 07, and 08.

E. Grad-CAM Record

Each CAM:

```

shape: (64,64,64)
dtype: float32
coordinate order: Z,Y,X
space: candidate_patch

```

Example metadata:

```

{
    "candidate_id": 17,

    "head": "malignancy",

    "coordinate_order": "ZYX",

    "gradcam_space": "candidate_patch",

    "shape_zyx": [
        64,
        64,
        64
    ],

    "local_center_zyx": [
        31.5,
        31.5,
        31.5
    ],

    "value_range": "[0,1]",

    "target_layer": "model.backbone.layer3[-1].conv3",

    "source_patch": "candidate_17.npy",
}

```

```
"projection_authority": "stage08"
}
```

3. Recommended Storage and Serialization

The optimal design is a **hybrid format**, rather than trying to put everything into JSON.

A. Large numerical arrays

Use:

NumPy `.npy`

for:

- CT volumes
- masks
- classifier patches
- Grad-CAM volumes

Example:

```
02_masked/
  ct_hu.npy
  lung_mask.npy

05_classifier_patches/
  patches/
    candidate_0001.npy
    candidate_0002.npy

07_gradcam/
  candidate_0001/
    gradcam/
      malignancy.npy
      spiculation.npy
```

Why `.npy`

It is:

- lossless,
 - fast,
 - memory-map capable,
 - trivial to load in Python,
 - ideal for fixed-shape arrays,
 - directly compatible with PyTorch and NumPy.
-

B. Metadata

Use:

JSON

for:

- coordinates,
- spacing,
- detector scores,
- model versions,
- patch geometry,
- classifier predictions,
- pipeline provenance.

Example:

```
metadata.json
classification.json
gradcam_summary.json
```

JSON should never contain large image arrays.

C. Batch Training

For training, thousands of individual `.npy` files can eventually create filesystem overhead.

For large-scale training, I recommend:

Option 1 — Zarr

Best for:

- large volumetric datasets,
- chunked CT access,
- distributed training,
- cloud/object storage,
- partial reads.

Structure:

```
training.zarr/
  ct/
  patches/
  labels/
  metadata/
```

For example:

```
patches.shape  
=  
(N, 64, 64, 64)
```

Labels:

```
labels.shape  
=  
(N, 8)
```

Candidate metadata:

```
candidate_id  
patient_id  
center_z  
center_y  
center_x  
...
```

This would be my preferred format for a future large-scale LungInsight training pipeline.

Option 2 — HDF5

Suitable when:

- training is primarily local,
- dataset structure is stable,
- one-file dataset storage is useful.

Example:

```
lunginsight_training.h5  
  
/patches  
/labels  
/candidate_metadata
```

However, Zarr is generally more flexible for parallel and cloud-oriented workflows.

D. Real-Time Inference

For inference, keep the pipeline simple.

Recommended:

DICOM

↓
in-memory NumPy arrays
↓
PyTorch tensors
↓
temporary or persisted .npy outputs
↓
JSON metadata

Avoid:

DICOM
→ HDF5
→ conversion
→ temporary dataset
→ PyTorch

for single-patient inference.

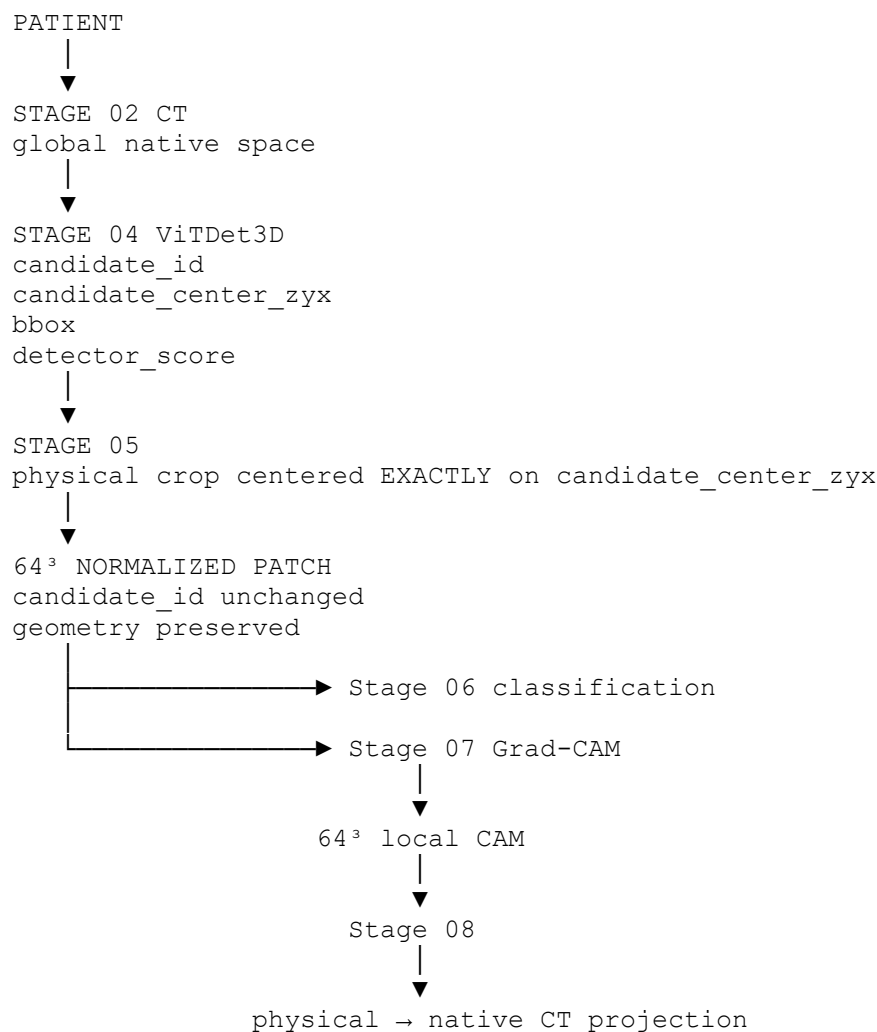
The inference storage layout should be:

```
output/
├── LIDC-IDRI-0141/
│   ├── 00_ingestion/
│   │   └── metadata.json
│   ├── 01_preprocessed/
│   │   ├── ct_hu.npy
│   │   └── metadata.json
│   ├── 02_masked/
│   │   ├── ct.npy
│   │   ├── lung_mask.npy
│   │   └── metadata.json
│   ├── 03_detector_input/
│   │   └── metadata.json
│   ├── 04_candidates/
│   │   ├── candidates.json
│   │   └── detector_metadata.json
│   ├── 05_classifier_patches/
│   │   ├── patches/
│   │   │   ├── candidate_0001.npy
│   │   │   └── ...
│   │   └── patches.json
│   ├── 06_classification/
│   │   └── classification.json
│   └── 07_gradcam/
│       ├── candidate_0001/
│       │   ├── gradcam/
│       │   │   ├── calcification.npy
│       │   │   ├── malignancy.npy
│       │   │   └── ...
│       │   └── montages/
```

```
├── metadata.json
├── gradcam_summary.json
├── 08_visualization/
│   ├── overlays/
│   ├── projections/
│   └── report.json
```

Recommended Final Data Contract

The most important architectural decision is to establish one immutable candidate identity and geometry chain:



Architectural rule set

1. ViTDet3D remains the Stage 04 detector.
2. Stage 04 detector coordinates are authoritative for candidate location.
3. Stage 05 must always center the classifier patch on the detector center.
4. All spatial arrays use (z, y, x).

5. **Physical coordinates must explicitly declare their axis convention.**
6. **Stage 05 is the authority for classifier-patch geometry.**
7. **Stage 06 propagates geometry; it does not reconstruct it.**
8. **Stage 07 produces only local (64, 64, 64) Grad-CAM volumes.**
9. **Each classifier head receives an independent Grad-CAM computation.**
10. **Stage 08 alone performs local-to-global projection.**
11. **Numerical volumes use .npy during inference.**
12. **JSON stores metadata and provenance only.**
13. **Zarr is the preferred long-term storage format for large-scale batch training.**

This architecture preserves the strong **pretrained ViTDet3D detection stage** while making the downstream classifier, Grad-CAM, and visualization pipeline spatially deterministic and traceable.